

HFT Ops Interview Preparation

Probability / Brainteasers / Code

Gabriel Kinshuk

July 12, 2026

Contents

1	Math / Probability Theory	2
1.1	Combinatorics	2
1.1.1	Permutations	2
1.1.2	Combinations	2
1.1.3	Distributing Indistinguishable Objects	2
1.1.4	Inclusion-Exclusion	2
1.1.5	Select Worked Problems	3
1.2	Probability	4
1.2.1	Complement	4
1.2.2	Independence	4
1.2.3	Mutual Exclusivity	5
1.2.4	Select Worked Problems	5
1.3	Expected Value	8
1.3.1	Select Worked Problems	9
1.4	Conditional Probability & Bayes	12
1.4.1	Select Worked Problems	13
1.5	Distributions	15
1.6	Bernoulli Distribution	15
2	Brainteasers	16
3	Code	18
3.1	SQL Patterns (PostgreSQL)	18
3.2	Python Patterns	21
3.3	Algorithmic Paradigms	27
3.4	Recursion and Memoization	29
3.5	Heaps and Priority Queues	31

1 Math / Probability Theory

1.1 Combinatorics

1.1.1 Permutations

Given a problem with n total options, of which we want to choose k in a particular order, we can compute the solution as:

$$P(n, k) = \frac{n!}{(n - k)!}$$

Suppose a different situation occurs where there are repeats among the elements. If some elements are identical, we divide by the factorial of each element's frequency to avoid counting duplicate arrangements:

$$P_{\text{repetitions}} = \frac{n!}{n_1! \cdot n_2! \cdot \dots \cdot n_k!}$$

1.1.2 Combinations

For cases where the order among the selected elements *does not* matter then we compute the possibilities as:

$$C(n, k) = \frac{n!}{k! \times (n - k)!}$$

If we substitute in the equation for permutations, we can see that this is equivalent to the number of permutations, dividing out the cases of reordering the elements:

$$C(n, k) = \frac{P(n, k)}{k!}$$

1.1.3 Distributing Indistinguishable Objects

We model this as arranging n identical items (stars) and $k - 1$ dividers (bars) in a line. Each arrangement partitions the items into k bins. The problem reduces to choosing $k - 1$ positions for dividers among $n + (k - 1)$ total positions:

$$C(n + k - 1, k - 1) = \binom{n + k - 1}{k - 1}$$

1.1.4 Inclusion-Exclusion

When counting the number of unique elements (cardinality) in a union of sets, we find the sum by counting each set individually, then subtracting the overlap:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Which extends to a larger number of sets as:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

Note that after removing the overlap from each pairwise intersection, we had to add BACK in the triple intersection, which would have been subtracted too many times. This pattern follows for higher orders of sets, where the odd length items (singles, triples) are added and the even length items (doubles, quadruples) are subtracted.

1.1.5 Select Worked Problems

Problem 1.1

How many ways can you arrange the letters in the word BANANA?

Solution. There are $n = 6$ elements. Repeated items have frequency 3 and 2 (Ns and As). Then the number of arrangements are $\frac{6!}{3! \times 2!} = \frac{720}{12} = 60$.

Problem 1.2

You roll 2 dice. How many distinct outcomes are there? What if you can't tell the dice apart?

Solution. With two dice, there are $6 \times 6 = 36$ outcomes if we can tell the dice apart. If we *can't* tell the dice apart, then we can solve either logically or formulaically:

Logically, we can see that from the original 36 combinations, there are 6 outcomes where the dice are the same, leaving 30 outcomes using mixed pairs of dice. Of these, half are duplicate when we can't tell the dice apart, leaving 15 outcomes. We can then add the 6 double outcomes back in for a total of 21 outcomes.

Formulaically, we can solve using a similar approach, taking the sum of mixed pair outcomes and double outcomes as $C(6, 2) + C(6, 1) = 15 + 6 = 21$.

Lastly, we can consider this problem using the stars and bars approach, where we notice that there are $n = 2$ identical dice and 6 distinct bins with $k = 5$ dividers between them. Then we can count the outcomes by $C(2 + 6 - 1, 6 - 1) = C(7, 5) = 21$

Problem 1.3

How many ways can you distribute 8 identical balls into 3 distinct boxes?

Solution. This is a direct application of stars and bars with $n = 8$ and $k = 3$. The number of outcomes is $C(8 + 3 - 1, 3 - 1) = C(10, 2) = \frac{10 \times 9 \times 8 \times \dots \times 3}{2} = 45$.

Problem 1.4

A deck has 52 cards. How many 5-card hands contain at least one ace?

Solution. Since this solution asks for the number of hands containing at least one ace, it will be simpler to compute the complement. This is the number of hands containing no aces, we then subtract from the total number of 5 card hands: $C(52, 5) - C(48, 5) = 886,656$.

Problem 1.5

You have 5 books on a shelf. In how many ways can you choose 3 books and arrange them left-to-right on a new shelf?

Solution. This is simply $C(5, 3) \times P(3, 3) = \frac{5 \times 4}{3!} \times \frac{3!}{0!} = \frac{60 \times 3!}{3!} = 60$.

Problem 1.6

A pizza place offers 8 toppings. How many different pizzas can you make if you can choose any number of toppings (including none)?

Solution. We can solve this by taking the summation of $C(8, i)$ from $i = 0$ to $i = 8$:

$$Total\ outcomes = \sum_{i=0}^8 C(8, i) = 256$$

There is a more succinct solution we can find if we approach the problem seeing each topping as the binary choice of being either on or off of the pizza, then the total outcomes are $2^8 = 256$.

Problem 1.7

How many ways can 12 people be split into 3 groups of 4?

Solution. We choose each group independently with the remaining members of the pool, and then divide by the reorderings since the groups are indistinguishable:

$$\frac{C(12, 4) \times C(8, 4) \times C(4, 4)}{3!} = \frac{495 \times 70 \times 1}{6} = 34,650.$$

1.2 Probability

Definition 1.1 (Sample Space). The sample space is the set of all possible outcomes of an event.

Many probability problems require defining the sample as an initial step, often followed by identifying the favourable outcomes.

Definition 1.2 (Events). The subset of the sample space containing the outcomes of interest.

1.2.1 Complement

Sometimes, probability-based questions which require computing many cases can be solved by instead calculating the complementary case using the identity

$$P(A) = 1 - P(\neg A)$$

Such as the previous “at least one...” scenario, where we may need to calculate several individual subcases if we do not use the complement.

1.2.2 Independence

If the outcomes of two events do *not* affect each other, then the two events are independent. Formally:

$$P(A \cap B) = P(A) \times P(B)$$

A common occurrence is when choosing elements from a group. If we choose items *with* replacement, then each draw is independent. Without replacement the pool changes size and composition after each draw.

1.2.3 Mutual Exclusivity

On the other end, there are events that cannot both happen at the same time, then:

$$P(A \cup B) = P(A) + P(B)$$

We can note that this is really just an application of the inclusion-exclusion equation, $P(A \cup B) = P(A) + P(B) - P(A \cap B)$, where the intersection is zero.

1.2.4 Select Worked Problems

Problem 1.8

How many people do you need in a room for a $>50\%$ chance that two share a birthday?

Solution. Computing this directly is impractical, since there are many ways for multiple people to share a birthday. Instead, using the complement, we simply subtract the probability that *no* people share a birthday from 1:

$$P(\text{all different}) = \prod_{i=1}^n \frac{365 - i + 1}{365}$$

We multiply additional factors until the quantity drops *below* 0.5 (because then $1 - P$ is greater than 0.5). This turns out to be $n = 23$, where $P(\text{all different}) \approx 0.493$ and thus $P(\text{birthday is shared}) \approx 0.507$.

Problem 1.9

You flip a fair coin 3 times. What's the probability of getting exactly 2 heads?

Solution. Flipping a coin three times has eight outcomes. Of these, three involve exactly two heads, thus the probability is $\frac{3}{8} = 0.375$.

If we don't want to enumerate each of the eight options and count directly, we could use $C(3, 2) = 3$ to find the number of outcomes involving two heads.

Problem 1.10

Roll two dice. What's the probability the sum is 7?

Solution. There are 36 outcomes of rolling two dice. Of these, the pairs that equal 7 are $(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)$. Then the probability is $\frac{6}{36} \approx 0.167$.

Problem 1.11

You draw 2 cards from a standard 52-card deck (without replacement). What's the probability both are hearts?

Solution. Without replacement, the draws are dependent. Then, the probability is:

$$\frac{13}{52} \times \frac{12}{51} \approx 0.0588$$

Note that *both* the numerator and the denominator shrank after we removed the first card.

We could also solve this by counting, if we divide the number of ways to pick two hearts by the total number of ways to pick two cards:

$$\frac{C(13, 2)}{C(52, 2)} = \frac{78}{1326} \approx 0.0588$$

Problem 1.12

You flip a fair coin 5 times. What's the probability of getting at least one head?

Solution. We can use the complement, the probability of getting *no* heads, subtracted from 1:

$$1 - \frac{1}{2^5} = 0.96875$$

Problem 1.13

You roll a fair die twice. What's the probability the second roll is strictly greater than the first?

Solution. There are 36 total ways to roll two dice. If the first roll is a 1, then there are 5 ways to roll higher on the second die. Similarly, there are 4 ways to roll higher if we rolled a 2, and so on.

Thus the number of favourable outcomes $1 + 2 + \cdots + 5$ or the triangle number of 5, $T_5 = 15$. The final answer is:

$$\frac{15}{36} \approx 0.417$$

Alternatively, we could solve by symmetry, calling roll 1 and 2 as A and B , respectively. If we notice that $P(A > B) = P(B < A)$, and $P(A = B) = \frac{6}{36}$, then we evenly divide the remaining outcomes by 2 to get $\frac{15}{36}$.

Problem 1.14

A box has 5 white and 3 black balls. You draw 3 without replacement. What's the probability of drawing exactly 2 white and 1 black?

Solution. The total ways to draw 3 balls are $C(8, 3) = 56$. We then find the ways to choose 2 white balls and 1 black ball individually, which is $C(5, 2)$ and $C(3, 1)$, then the answer is:

$$\frac{C(5, 2) \times C(3, 1)}{C(8, 3)} = \frac{10 \times 3}{56} \approx 0.536$$

Since we remove the balls without replacement, this problem follows the hypergeometric distribution.

Problem 1.15

You randomly pick a number from 1 to 100. What's the probability it's divisible by 3 or 5?

Solution. There are 33 numbers divisible by 3, and 20 numbers divisible by 5. The numbers divisible by both are the multiples of 15 of which there are 6 (15, 30, 45, 60, 75, 90). Thus:

$$P_{Div}(A \cup B) = \frac{33}{100} + \frac{20}{100} - \frac{6}{100} = 0.47$$

Problem 1.16

A family has 3 children. Assuming each child is equally likely to be a boy or girl, what's the probability that all 3 are the same gender?

Solution. There are eight outcomes, of those there are two in which all children are the same gender, so the answer is $\frac{2}{8} = 0.25$.

Problem 1.17

You throw 3 dice. What's the probability ALL three show different numbers?

Solution. There are $6^3 = 216$ total ways to throw three dice. After we throw the first dice, there are 5 options remaining, and 4 after the second dice. Then the answer is:

$$\frac{6 \times 5 \times 4}{6^3} = \frac{120}{216} \approx 0.556$$

Problem 1.18

You toss a fair coin until you get heads. What's the probability you need exactly 3 tosses?

Solution. We stop at the first heads, so needing exactly 3 tosses means the first two tosses are tails and the third is heads: T, T, H. Since tosses are independent and each outcome has probability $\frac{1}{2}$,

$$P(\text{exactly 3}) = \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{8}.$$

We do not enumerate anything else, because exactly one sequence produces this event.

Remark (Two ways probability gets counted). The reason we can pin down this single case without listing every outcome is that this problem lives in a different kind of sample space than counting problems like the birthday question.

When every outcome is equally likely, probability is combinatorial. You divide favorable outcomes by total outcomes, so you have to characterize the entire sample space. Each outcome carries no probability on its own and earns weight only by being counted against the total.

When outcomes are not equally likely, each outcome carries its own probability directly, usually a product of independent step probabilities. Here the sequence TTH has probability $\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{8}$ by itself, with no denominator to divide by. That is why one case can be solved in isolation.

These are the same axioms applied to different spaces. Counting favorable over total is the special case that appears when all per-outcome probabilities are equal, so the sum of probabilities collapses into a plain count. The coin problem is the general version, where the per-outcome probabilities differ and you take the one you want.

Problem 1.19

In a room of 5 people, what's the probability that at least two share a birthday? (Assume 365 days, equally likely.)

Solution. Again, we use the complement to find 1 minus the probability that no one shares a birthday:

$$1 - \left(\frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \frac{362}{365} \times \frac{361}{365} \right) \approx 1 - 0.9729 \approx 0.0271$$

1.3 Expected Value

Definition 1.3 (Expected Value). The long-run average outcome of a random process:

$$\mathbb{E}[X] = \sum x_i \times P(x_i)$$

Which is simply the weighted average of all possible outcomes. A note is that the EV is neither required to be a possible outcome (e.g., the EV of a dice roll is 3.5) nor the most likely outcome.

Theorem 1.4 (Linearity of Expectation). *The EV of multiple events is the sum of their EVs:*

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$$

This holds always, even if X and Y are dependent.

Definition 1.5 (Indicator Random Variables). A variable I_A that equals 1 when event A occurs and 0 otherwise. Its expectation is just the event's probability, since $\mathbb{E}[I_A] = 1 \cdot P(A) + 0 \cdot P(A^c) = P(A)$.

The point is not the variable itself, which is trivial, but the decomposition it enables. To find the expected *count* of something, write that count as a sum of indicators, one per event, then apply linearity:

$$\mathbb{E} \left[\sum_i I_{A_i} \right] = \sum_i P(A_i).$$

This turns a hard count with an ugly distribution into a sum of easy probabilities, and because linearity ignores dependence, it works even when the events are entangled.

Theorem 1.6 (Law of Total Expectation). *For any partition of the sample space into disjoint cases $B_1, \dots, B_n$ with $\sum_i P(B_i) = 1$,*

$$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[X \mid B_i] \cdot P(B_i).$$

This is the definition of expected value regrouped by cases: instead of averaging over individual outcomes, average the conditional mean within each case, weighted by how likely that case is. Mathematically it adds nothing new, since expanding each $\mathbb{E}[X \mid B_i]$ recovers the flat sum. Operationally it is a different tool, useful in three situations:

- *when you are handed conditional averages and case probabilities rather than the outcome distribution*
- *when X is continuous, so the flat sum $\sum_x x P(X = x)$ does not apply but conditioning on discrete cases still does;*

- when $\mathbb{E}[X \mid B_i]$ refers back to $\mathbb{E}[X]$ itself, turning the conditioning into a solvable equation

The cases must be mutually exclusive and cover everything. Overlapping or incomplete cases break the weighting.

Example 1.7 (Conditioning on cases). A commute depends on weather. Sunny days are 60% of days with a 20 minute average; rainy days are 40% with a 35 minute average. The two cases partition every day, so

$$\mathbb{E}[\text{commute}] = 0.6(20) + 0.4(35) = 12 + 14 = 26 \text{ minutes.}$$

Example 1.8 (Recursive EV). Suppose we were interested in the expected number of rolls we would take if we continue rolling a die until we hit a 6. If we call E the expected number of rolls, then:

$$E = 1 + \frac{5}{6} \cdot E \longrightarrow E - \frac{5}{6} \cdot E = 1 \longrightarrow E = 6$$

We can note that this is the EV of the geometric distribution, the expected trials until first success with probability $\frac{1}{p}$.

1.3.1 Select Worked Problems

Problem 1.20

A game costs \$1. You flip a coin: heads you win \$3, tails you win nothing. What's $\mathbb{E}[\text{Profit}]$?

Solution. $EV = 0.5 \cdot 2 + 0.5 \cdot -1 = +0.5$

Problem 1.21

Roll a die. If it shows 6, you roll again and add. Otherwise, your score is the number shown. What's the expected score?

Solution.

$$EV = 3.5 + \frac{1}{6} \cdot EV \longrightarrow \frac{5}{6} \cdot EV = 3.5 \longrightarrow EV = 4.2$$

Problem 1.22

Shuffle a standard deck. What's the expected number of cards that are in their "correct" position (card k is in position k)?

Solution. This is linearity of expectation applied to indicators. Let $I_k = 1$ if card k lands in position k , and 0 otherwise. In a random shuffle each card is equally likely to occupy any of the 52 positions, so $\mathbb{E}[I_k] = P(I_k = 1) = \frac{1}{52}$. The number of correctly placed cards is $M = \sum_{k=1}^{52} I_k$, and by linearity

$$\mathbb{E}[M] = \sum_{k=1}^{52} \mathbb{E}[I_k] = 52 \cdot \frac{1}{52} = 1.$$

The I_k are dependent, but linearity does not require independence, so that never matters.

Problem 1.23

You flip a coin until you get heads. How many flips do you expect?

Solution.

$$EV = 1 + 0.5 \cdot EV \longrightarrow 0.5 \cdot EV = 1 \longrightarrow EV = 2$$

Problem 1.24

A cereal box contains 1 of 4 different toys (equally likely). How many boxes must you buy to expect to collect all 4?

Solution. The total number of boxes is a sum of four phase waits, so we use linearity of expectation. In phase k we already hold $k - 1$ toys and buy boxes until a new one appears. The per-trial success probability is $p = \frac{\text{favourable}}{\text{total}}$, the number of missing toys over 4. We want the reciprocal, the expected number of trials per success, which is $\frac{1}{p}$:

$$\begin{aligned} 0 \text{ toys, } p &= \frac{4}{4} \longrightarrow \mathbb{E}[T_1] = \frac{1}{p} = 1 \\ 1 \text{ toy, } p &= \frac{3}{4} \longrightarrow \mathbb{E}[T_2] = \frac{1}{p} = \frac{4}{3} \\ 2 \text{ toys, } p &= \frac{2}{4} \longrightarrow \mathbb{E}[T_3] = \frac{1}{p} = 2 \\ 3 \text{ toys, } p &= \frac{1}{4} \longrightarrow \mathbb{E}[T_4] = \frac{1}{p} = 4 \end{aligned}$$

$$\mathbb{E}[T] = \mathbb{E}[T_1] + \mathbb{E}[T_2] + \mathbb{E}[T_3] + \mathbb{E}[T_4] = 1 + \frac{4}{3} + 2 + 4 = \frac{25}{3} \approx 8.33$$

Problem 1.25

I roll a die repeatedly and keep a running sum. What's the expected number of rolls until the sum first reaches or exceeds 10?

Solution. We solve recursively. Let $\mathbb{E}[s]$ be the expected number of additional rolls needed to first reach a sum of at least 10, starting from current sum s . The answer we want is $\mathbb{E}[0]$.

Base case: $\mathbb{E}[s] = 0$ for all $s \geq 10$, since the target is already met. For $s < 10$, one roll costs 1 and moves to $s + 1, \dots, s + 6$ with equal probability $\frac{1}{6}$:

$$\mathbb{E}[s] = 1 + \frac{1}{6} \sum_{j=1}^6 \mathbb{E}[s + j], \quad \mathbb{E}[s] = 0 \text{ for } s \geq 10.$$

Working downward from the base case:

$$\begin{aligned} \mathbb{E}[9] &= 1 + \frac{1}{6}(\mathbb{E}[10] + \dots + \mathbb{E}[15]) = 1 \\ \mathbb{E}[8] &= 1 + \frac{1}{6}\mathbb{E}[9] = \frac{7}{6} \\ \mathbb{E}[7] &= 1 + \frac{1}{6}(\mathbb{E}[8] + \mathbb{E}[9]) = \frac{49}{36} \\ &\vdots \\ \mathbb{E}[0] &\approx 3.6 \end{aligned}$$

Problem 1.26

You roll a fair six-sided die. After seeing the result, you may keep it as your score, or reroll. If you reroll, the new value replaces the old one (you cannot go back to a discarded roll). You are allowed up to three rolls total, and your score is the value you are holding when you stop. Playing optimally to maximize your expected score, what is that expected score, and what is the stopping rule at each stage?

Solution. Let $\mathbb{E}[r]$ be the expected score with r rerolls remaining. Solve backwards.

With no rerolls left, you keep whatever you roll, so $\mathbb{E}[0] = 3.5$.

With one reroll left, you keep the current face only if it beats the continuation value $\mathbb{E}[0] = 3.5$, i.e. keep on $\{4, 5, 6\}$ and reroll otherwise:

$$\mathbb{E}[1] = \frac{1}{6}(4) + \frac{1}{6}(5) + \frac{1}{6}(6) + \frac{3}{6}(3.5) = \frac{17}{4} = 4.25.$$

With two rerolls left, the continuation value is now $\mathbb{E}[1] = 4.25$, so you keep only faces that beat it, $\{5, 6\}$, and reroll on $\{1, 2, 3, 4\}$:

$$\mathbb{E}[2] = \frac{1}{6}(6) + \frac{1}{6}(5) + \frac{4}{6}(4.25) = \frac{14}{3} \approx 4.67.$$

Problem 1.27

You pick a random number uniformly from $[0, 1]$. Then you pick another. Then another. You stop when the running total exceeds 1. How many numbers do you expect to pick?

Solution. Let N be the number of draws until the running sum first exceeds 1, and let $X_1, X_2, \dots$ be the draws. We want $\mathbb{E}[N]$.

Step 1: a tail probability. The *tail probability* $\mathbb{P}(N > n)$ is the chance N exceeds a cutoff n , i.e. the chance we still need more draws after n of them. Needing more than n draws means the first n have not yet crossed 1:

$$N > n \iff X_1 + \dots + X_n \leq 1.$$

Step 2: geometry The point $(X_1, \dots, X_n)$ lands uniformly in the unit hypercube $[0, 1]^n$, which has volume 1. Since the draws are uniform, the probability of landing in any region equals that region's volume. The region where the coordinates are nonnegative and sum to at most 1 is a *simplex* (the n -dimensional triangle: a segment in 1D, triangle in 2D, tetrahedron in 3D). Its volume is $\frac{1}{n!}$ ($V_n(t) = \int_0^t V_{n-1}(t-x) dx$, starting from $V_1(t) = t$. Then $V_n(t) = \frac{t^n}{n!}$, evaluated at $t = 1$, $V_n(1) = \frac{1}{n!}$), so

$$\mathbb{P}(N > n) = \mathbb{P}(X_1 + \dots + X_n \leq 1) = \frac{1}{n!}.$$

Step 3: sum the tails. For a nonnegative integer variable, the mean equals the sum of its tail probabilities:

$$\mathbb{E}[N] = \sum_{n=0}^{\infty} \mathbb{P}(N > n).$$

This holds because an outcome $N = k$ clears the cutoffs $n = 0, 1, \dots, k-1$, so it is counted exactly k times across the sum, contributing its own value. Substituting the tails:

$$\mathbb{E}[N] = \sum_{n=0}^{\infty} \frac{1}{n!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \dots = e \approx 2.718.$$

1.4 Conditional Probability & Bayes

Definition 1.9 (Conditional Probability). The probability of A , given that B has already happened:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Theorem 1.10 (Bayes' Theorem). Baye's theorem allows us to update the probability of a hypothesis as more information becomes available:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Where:

- $P(A|B)$ is the posterior probability
- $P(B|A)$ is the likelihood
- $P(A)$ is the prior probability
- $P(B)$ is the marginal probability

Theorem 1.11 (Law of Total Probability). If $B_1, B_2, \dots, B_n$ partition the sample space (are mutually exclusive and exhaustive), then:

$$P(A) = \sum P(A|B_i) \times P(B_i)$$

We often use this to compute $P(B)$ when we don't know it directly, but know both $P(B|A)$ and $P(B|\neg A)$.

Example 1.12 (Monty Hall Problem). Assume the contestant chooses Door 1 and Monty opens Door 2 to reveal a goat. Let A_i be the event the car is behind Door i (prior $P(A_i) = \frac{1}{3}$), and B be the event Monty opens Door 2.

The likelihoods $P(B|A_i)$ are:

- $P(B|A_1) = \frac{1}{2}$ (Monty randomly picks Door 2 or 3)
- $P(B|A_2) = 0$ (Monty cannot reveal the car)
- $P(B|A_3) = 1$ (Monty is forced to open Door 2)

The marginal probability of B is:

$$P(B) = \sum_{i=1}^3 P(B|A_i)P(A_i) = \left(\frac{1}{2}\right)\left(\frac{1}{3}\right) + (0)\left(\frac{1}{3}\right) + (1)\left(\frac{1}{3}\right) = \frac{1}{2}$$

By Bayes' Theorem, the posterior probabilities of winning by staying (A_1) or switching (A_3) are:

$$P(A_1|B) = \frac{P(B|A_1)P(A_1)}{P(B)} = \frac{\left(\frac{1}{2}\right)\left(\frac{1}{3}\right)}{\frac{1}{2}} = \frac{1/6}{1/2} = \frac{1}{3}$$
$$P(A_3|B) = \frac{P(B|A_3)P(A_3)}{P(B)} = \frac{(1)\left(\frac{1}{3}\right)}{\frac{1}{2}} = \frac{1/3}{1/2} = \frac{2}{3}$$

Switching doubles the probability of winning.

1.4.1 Select Worked Problems

Problem 1.28

I roll two dice. Given that the sum is 8, what's the probability that one of the dice shows a 3?

Solution. The possible ways to roll a sum of 8 are: (2, 6), (3, 5), (4, 4), (5, 3), (6, 2). Of these, two options have a 3 in them, so the answer is $\frac{2}{5} = 0.4$.

Problem 1.29

A bag has 3 red and 2 blue balls. You draw one, don't replace it, then draw another. What's $P(\text{2nd is red} \mid \text{1st was blue})$?

Solution. Once we drew one blue on, the remaining bag composition is 3 red balls and 1 blue one. Then, there is a $\frac{3}{4}$ probability that the second will be red.

Problem 1.30

60% of emails are spam. A filter catches 95% of spam but also flags 3% of legitimate emails. An email is flagged. What's the probability it's actually spam?

Solution. We want $P(\text{Spam} \mid \text{Flagged})$. The likelihood $P(\text{Flagged} \mid \text{Spam})$ is given as 0.95, the prior $P(\text{Spam})$ is 0.6, and the marginal probability $P(\text{Flagged})$ is $0.95 \cdot 0.6 + 0.03 \cdot 0.4 = 0.582$.

Then the posterior is

$$P(\text{Spam} \mid \text{Flagged}) = \frac{P(\text{Flagged} \mid \text{Spam}) \times P(\text{Spam})}{P(\text{Flagged})} = \frac{0.95 \times 0.6}{0.582} \approx 0.979$$

Problem 1.31

You have 2 coins. Coin A is fair (50% heads). Coin B is biased (75% heads). You pick a coin at random and flip it. It lands heads. What's the probability you picked coin B?

Solution. We are looking for the posterior, $P(B \mid H)$.

The priors are $P(A) = P(B) = 0.5$. The likelihoods are $P(H \mid A) = 0.5$ and $P(H \mid B) = 0.75$. Then $P(H) = P(H \mid A) \cdot P(A) + P(H \mid B) \cdot P(B) = 0.5 \cdot 0.5 + 0.75 \cdot 0.5 = 0.625$. Then finally

$$P(B \mid H) = \frac{P(H \mid B) \cdot P(B)}{P(H)} = \frac{0.75 \cdot 0.5}{0.625} = 0.6$$

After flipping and landing on heads, our belief that we picked coin B increased from 0.5 to 0.6.

Problem 1.32

Same as 1.31. The chosen coin is flipped twice and get heads both times. Now what's $P(B)$?

Solution. The only difference is our priors are $P(A) = 0.4$ and $P(B) = 0.6$. Then we update $P(H) = P(H \mid A) \cdot P(A) + P(H \mid B) \cdot P(B) = 0.5 \cdot 0.4 + 0.75 \cdot 0.6 = 0.65$. Finally:

$$P(B \mid H) = \frac{P(H \mid B) \cdot P(B)}{P(H)} = \frac{0.75 \cdot 0.6}{0.65} = \frac{9}{13} \approx 0.692$$

Problem 1.33

A family has 2 children. Given that at least one child is a boy, what's the probability both are boys?

Solution. We can answer this by considering the restriction over the sample space caused by the new information. Originally, for two children there would be four possibilities, of which 1 has two boys. After we know that at least one child is a boy, only three options remain $(B, G), (B, B), (G, B)$. Thus, the probability both are boys is $\frac{1}{3}$.

Problem 1.34

Three cards: one is red on both sides, one is blue on both sides, one is red on one side and blue on the other. You pick a card at random and see a red side. What's the probability the other side is also red?

Solution. The natural first instinct is to treat the cards as the sample space, as with the births problem: restrict to cards consistent with seeing a red face, then count outcomes among those. This fails here because a card is not a single unit of observation. When a card has two sides of the same color, "seeing red" can happen in more than one way for that card. There's no such internal multiplicity in the births problem.

So take the six sides, not the three cards, as the equally likely outcomes. Three of these six are red: two belong to the RR card, one belongs to the RB card. Since each red side was equally likely to be the one we observed, and two of the three red sides have a red back:

$$P(\text{other side red} \mid \text{observed red}) = \frac{2}{3}.$$

Problem 1.35

You have a 6-sided die. You don't know if it's fair or loaded. A fair die shows each number with probability $1/6$. The loaded die shows 6 with probability $1/2$ and each other number with probability $1/10$. You pick a die at random (50-50) and roll a 6. What's $P(\text{loaded})$?

Solution. By Bayes' theorem, with $P(\text{loaded}) = P(\text{fair}) = 0.5$:

$$P(\text{loaded} \mid 6) = \frac{P(6 \mid \text{loaded}) P(\text{loaded})}{P(6)} = \frac{0.5 \times 0.5}{0.5 \times 0.5 + \frac{1}{6} \times 0.5} = \frac{0.25}{1/3} = 0.75$$

Alternatively, work in odds instead of probabilities. Dividing the Bayes' theorem expressions for $P(\text{loaded} \mid 6)$ and $P(\text{fair} \mid 6)$ cancels $P(6)$ from both, leaving:

$$\underbrace{\frac{P(\text{loaded} \mid 6)}{P(\text{fair} \mid 6)}}_{\text{posterior odds}} = \underbrace{\frac{P(6 \mid \text{loaded})}{P(6 \mid \text{fair})}}_{\text{likelihood ratio}} \times \underbrace{\frac{P(\text{loaded})}{P(\text{fair})}}_{\text{prior odds}}$$

So the posterior odds are just the prior odds scaled by the likelihood ratio, no need to compute $P(6)$ at all. Here, the prior odds are $\frac{P(\text{loaded})}{P(\text{fair})} = \frac{0.5}{0.5} = 1$, and the likelihood ratio is $\frac{P(6 \mid \text{loaded})}{P(6 \mid \text{fair})} = \frac{0.5}{1/6} = 3$. So the posterior odds are $1 \times 3 = 3$, i.e. $3 : 1$ in favor of loaded, giving $P(\text{loaded} \mid 6) = 3/4 = 0.75$.

In short, a six is three times more likely to appear if the die is loaded, so each 6 observed multiplies the odds by 3. If we observed *another* 6, the odds would update again: $3 : 1 \rightarrow 9 : 1$, giving $P(\text{loaded}) = \frac{9}{10}$.

1.5 Distributions

Definition 1.13 (Distribution). A distribution is a complete specification of how probability is spread over the possible outcomes of a random variable: a mapping from each outcome (or range of outcomes, in the continuous case) to its probability, with the total summing to 1.

Pattern recognition over derivation

A *named* distribution (Binomial, Geometric, Poisson, etc.) is a family of distributions arising from a common underlying random process, indexed by one or more parameters (e.g. n, p for Binomial). Once a problem is identified as an instance of that process, its PMF/PDF, mean, and variance follow immediately from the known formulas for the family, without re-derivation.

1.6 Bernoulli Distribution

Definition 1.14 (Bernoulli). A random variable X follows a Bernoulli distribution with parameter $p \in [0, 1]$, written $X \sim \text{Bernoulli}(p)$, if it takes exactly two values:

$$P(X = 1) = p, \quad P(X = 0) = 1 - p.$$

It models a single trial with two outcomes, conventionally labeled success (1) and failure (0).

$$\mu = p \quad \sigma^2 = p(1 - p)$$

Building block for other distributions

The Bernoulli is not typically the distribution of interest on its own, it is the atomic unit other named distributions are built from. A Binomial random variable is a sum of n i.i.d. $\text{Bernoulli}(p)$ trials; a Geometric random variable counts trials until the first Bernoulli success. Recognizing a Bernoulli inside a larger problem, often via an indicator variable $\mathbf{1}\{\cdot\}$, is frequently the first step in applying linearity of expectation to more complex sums.

2 Brainteasers

Problem 2.1 [Easy]

Five pirates need to divide 100 Gold Coins. Pirates have a hierarchy, from Level 1 to level 5. The highest level pirate is the leader. The leader proposes a plan to distribute the gold and all the pirates vote on it (including the leader). If at least 50% of the pirates agree on the plan, the gold is split according to the proposal. If not, level-5 pirate is kicked from the ship, and the level-4 pirate now proposes a new plan. This process continues until a proposal is accepted. All pirates are extremely smart and extremely greedy. How does level-5 Pirate divide the treasure?

Solution. We approach this problem game-theoretically and dynamically:

- 2 pirates → Pirate 2 proposes 100 coins for themselves and zero for Pirate 1
- 3 pirates → Pirate 3 bribes Pirate 1 with one coin and keeps 99 for themselves, which they accept because they get zero coins if Pirate 3 is booted off.
- 4 pirates → Pirate 4 bribes Pirate 2 with one coin and keeps 99 for themselves, in the same manner as the 3-pirate case.
- 5 pirates → Pirate 5 bribes pirates 1 and 3 with one coin each, keeping 98 for themselves.

Thus, the final division is [98, 0, 1, 0, 1].

Problem 2.2 [Easy]

An explorer travels 1 mile South, 1 mile East, then 1 mile North, and finds themselves back at the starting point. The explorer spots a bear, what is the color of the bear?

Solution. The bear must be white. There are a few relevant scenarios that could have resulted in the explorer starting back where they began. In the first case, they started at the North pole, then travelling South equates to travelling directly away from the Pole. They can then travel *any* distance East or West, the following mile back North will return them to their destination. There are no bears in the South Pole so we omit this, though the same logic applies (with the Eastward movement equivalent to spinning in place or not moving at all).

There are other solutions near the South Pole. If the traveler's initial one-mile journey South ends on a latitude circle with a circumference of $\frac{1}{n}$ ($n \in \mathbb{Z}^+$) miles, walking one mile East will complete n full rotations around the pole and leave them exactly where they started walking East, meaning the following mile North would place them in the same starting position.

Problem 2.3 [Easy]

There is a light bulb inside a room and three switches outside. All switches are currently in off state and only one switch controls the light bulb. You may turn any number of switches on or off any number of times you want. How many times do you need to go into the room to figure out which switch controls the light bulb?

Solution. We only need to enter the room once. The first switch is flipped and left on for enough time that the bulb becomes hot. It is then flipped off and the second switch is flipped, we then immediately enter the room. This leads to three cases:

1. Bulb is hot → the light must be off and the first switch controls the light

2. Bulb is on $\rightarrow$ the second switch controls the light
3. Bulb is off and cold $\rightarrow$ the third switch controls the light

If there were four switches, we can still infer the switch in one entry by turning on both switches 1 and 2 for a long time, then switching off switch 2 and turning on switch 3 before entering the room. This results in the following cases:

1. Bulb is on and hot $\rightarrow$ Switch 1 controls the light
2. Bulb is on and cool $\rightarrow$ Switch 3 controls the light
3. Bulb is off and hot $\rightarrow$ Switch 2 controls the light
4. Bulb is off and cool $\rightarrow$ Switch 4 controls the light

The extension to four bulbs worked because we had two binary variables (illumination and bulb temperature) which creates 4 distinct states, of which we were only using 3 in the 3-bulb case.

Problem 2.4 [*Easy*]

How do you place 50 good candies and 50 rotten candies in two boxes such that if you choose a box at random and take out a candy at random, it better be good!

We need to maximize the probability of getting a good candy when selecting a random box and a random candy from it.

Solution. On one extreme, we can place all of the good candies in one box with all of the bad candies in another. This gives us $\mathbb{E}[G] = 0.5$. Thus, we should find solutions that better than 50%. We can beat this rather simply, by placing one good candy in one box and the other 49 in the other box with all of the bad candies. This results in:

$$\mathbb{E}[G] = 0.5 * \frac{1}{1} + 0.5 * \frac{49}{99} \approx 0.747$$

3 Code

3.1 SQL Patterns (PostgreSQL)

Reference for recurring query shapes. Each entry states when to reach for it and the mechanism that makes it work.

Logical processing order

A query is evaluated in this order, regardless of written order:

FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

Two consequences drive most of the patterns below. **WHERE** runs before aggregation, so it cannot see aggregates or **SELECT** aliases. Window functions are computed at the **SELECT** step, so they also cannot appear in **WHERE** and must be wrapped in a subquery or CTE to filter on.

WHERE vs HAVING. **WHERE** filters individual rows before grouping. **HAVING** filters whole groups after aggregation and is the only place an aggregate predicate is legal. A filter on a plain (non-aggregated) column belongs in **WHERE** so rows are dropped before the grouping work happens. Reserve **HAVING** for conditions like `COUNT(*) > 5`.

```
1 SELECT department, COUNT(*) AS headcount
2 FROM employees
3 WHERE active = true           -- row filter, before grouping
4 GROUP BY department
5 HAVING COUNT(*) > 5;         -- group filter, on an aggregate
```

Listing 1: WHERE prunes rows, HAVING prunes groups

Joins.

- **INNER:** rows with a match on both sides.
- **LEFT / RIGHT:** keep all rows from one side, NULL-fill the other.
- **FULL OUTER:** keep unmatched rows from both sides.
- **CROSS:** Cartesian product, every left row against every right row.
- **SELF:** a table joined to itself under an alias (employee to manager, row to prior row).
- **Semi-join** (EXISTS, IN): keep left rows with at least one match. No row duplication, no right-side columns.
- **Anti-join** (NOT EXISTS, or `LEFT JOIN ... WHERE right.key IS NULL`): keep left rows with no match.

Rank-then-filter with ROW_NUMBER(). Use this for top-*N* per group when you need to keep the full row. **GROUP BY** would collapse the other columns away. Number rows inside each partition, then filter on that number in an outer query, since the window function cannot live in **WHERE**.

Choose the ranker by tie policy: **ROW_NUMBER** assigns distinct numbers and breaks ties arbitrarily, **RANK** repeats a rank and leaves gaps, **DENSE_RANK** repeats with no gaps.

```

1 SELECT department, name, salary
2 FROM (
3     SELECT department, name, salary,
4           ROW_NUMBER() OVER (PARTITION BY department
5                               ORDER BY salary DESC) AS rn
6     FROM employees
7 ) ranked
8 WHERE rn = 1;

```

Listing 2: Highest-paid employee per department

Gaps and islands. An *island* is a maximal run of consecutive values. The trick: subtract a dense row number from the value. Across a consecutive run both sides increase by one per step, so the difference stays constant, and it jumps exactly at a gap. Group by the difference to collapse each run.

```

1 WITH numbered AS (
2     -- DISTINCT first so duplicate ids do not desync the row number
3     SELECT DISTINCT log_id,
4           ROW_NUMBER() OVER (ORDER BY log_id) AS rn
5     FROM logs
6 )
7 SELECT MIN(log_id) AS island_start,
8        MAX(log_id) AS island_end,
9        COUNT(*)    AS length
10 FROM numbered
11 GROUP BY log_id - rn
12 ORDER BY island_start;

```

Listing 3: Islands of consecutive ids

The DISTINCT is load-bearing. If `log_id` repeats, `rn` still advances on the duplicate while the value does not, so `log_id - rn` shifts and splits an island that should stay whole. For consecutive *dates* rather than integers, offset by an interval: `log_date - rn * INTERVAL '1 day'` is constant within a run.

```

1 SELECT log_id + 1 AS missing_from,
2        next_id - 1 AS missing_to
3 FROM (
4     SELECT log_id,
5           LEAD(log_id) OVER (ORDER BY log_id) AS next_id
6     FROM logs
7 ) t
8 WHERE next_id - log_id > 1;    -- values missing between the two present
                                ids

```

Listing 4: Gaps via LEAD

Conditional aggregation (pivot). Fold a categorical column into one row of counts by wrapping a CASE in an aggregate. Rows the CASE does not match contribute 0 to SUM. In Postgres the FILTER clause does the same thing and reads more cleanly.

```
1 SELECT
2     inspection_type,
3     COUNT(*) FILTER (WHERE risk_category IS NULL)           AS no_risk,
4     COUNT(*) FILTER (WHERE risk_category = 'Low Risk')      AS low_risk
5     ,
6     COUNT(*) FILTER (WHERE risk_category = 'Moderate Risk') AS
7     moderate_risk,
8     COUNT(*) FILTER (WHERE risk_category = 'High Risk')     AS
9     high_risk,
10    COUNT(*)                                                AS total
11 FROM sf_restaurant_health_violations
12 GROUP BY inspection_type
13 ORDER BY total DESC;
```

Listing 5: Pivot risk categories to columns

The SUM(CASE WHEN ... THEN 1 ELSE 0 END) form is equivalent and portable across engines. COUNT ignores NULL, so COUNT(CASE WHEN cond THEN 1 END) (no ELSE) also counts only the matches.

Gotchas.

- **NOT IN with NULLs:** if the subquery returns any NULL, `x NOT IN (...)` is never true and the query returns zero rows. `x NOT IN (1, NULL)` expands to `NOT (x=1 OR x=NULL)`, and the NULL comparison poisons the result to unknown. Use `NOT EXISTS`, which is null-safe.
- **Integer division truncates:** `5 / 2` yields 2 because the fractional part is discarded. Cast one operand: `5::numeric / 2` or `5.0 / 2`. Bites hardest in ratio and percentage columns.
- **COUNT(col) vs COUNT(*):** the former skips NULLs in col, the latter counts every row. A common source of off-by-*N* errors.

3.2 Python Patterns

Good solutions to interview problems are often determined by the choice of container and knowledge of which built-in primitives express the required operation.

Containers and cost profiles

Selecting a container is selecting a cost profile. That decision precedes the algorithm and frequently dominates the runtime.

	Ordered	Mutable	Lookup	Use
<code>list</code>	yes	yes	$O(n)$	ordered sequence, duplicates permitted
<code>tuple</code>	yes	no	$O(n)$	fixed record, or a dictionary key
<code>set</code>	no	yes	$O(1)$	membership, deduplication
<code>dict</code>	insertion	yes	$O(1)$	mapping from key to value

Membership cost

`x in some_list` is $O(n)$. `x in some_set` is $O(1)$ on average. A membership test inside a loop over n elements is therefore $O(n^2)$ against a list and $O(n)$ against a set. Converting the list to a set once, before the loop, removes a factor of n . This substitution accounts for a large share of naive-to-optimal improvements.

Hashability. Set elements and dictionary keys must be hashable, which in practice requires immutability. A `tuple` is a valid key. A `list` is not, since mutation would invalidate its position in the hash table. Converting a list to a tuple to form a composite key is therefore a standard construction, as in grouping anagrams by their sorted letters.

```
1 seen = set()
2 seen.add((row, col))           # legal, tuple is hashable
3 # seen.add([row, col])         # TypeError: unhashable type
```

Listing 6: Tuple as a composite key

Dictionary access. Counting, grouping, and seen-before problems reduce to a dictionary. Three access forms differ in their treatment of a missing key.

```
1 d[k]                           # raises KeyError if k is absent
2 d.get(k, default)              # returns default instead of raising
3 d.setdefault(k, [])           # inserts default if absent, then returns it
4
5 for k in d:                    # iterates over keys
6 for k, v in d.items():         # iterates over pairs
```

Listing 7: Dictionary access

Built-in functions

An accumulator loop is often a built-in written out longhand. The following cover most cases.

Built-in	Expresses
<code>len, sum, min, max</code>	size and aggregation
<code>sorted</code>	ordering, parameterized by key and reverse
<code>any(...)</code>	existential quantifier: at least one element satisfies p
<code>all(...)</code>	universal quantifier: every element satisfies p
<code>enumerate</code>	index paired with value
<code>zip</code>	parallel iteration over several sequences
<code>reversed, range</code>	traversal order
<code>abs, round, divmod</code>	numeric utilities

`any` and `all` short-circuit. The expression `any(x < 0 for x in xs)` terminates at the first negative value.

The key argument. `sorted`, `min`, and `max` accept a key function applied to each element. Ranking is performed on the image of that function while the original element is returned. The consequence is that a record can be ranked on one of its fields and recovered whole.

```
1 min(scores)                                # returns a number
2 min(records, key=lambda r: r[1])           # returns the full record
3
4 sorted(words, key=len)                     # rank by length
5 sorted(records, key=lambda r: (-r[1], r[0])) # score desc, name asc
```

Listing 8: Ranking on a field, returning the record

A tuple key is the analogue of a multi-column `ORDER BY`. Negating a single numeric field reverses that field alone. `reverse=True` cannot express this, since it inverts every field simultaneously.

Comprehensions

A comprehension expresses selection and projection in one expression. The form `[f(x) for x in xs if p(x)]` corresponds to selecting $f(x)$ over the elements for which $p(x)$ holds. Four bracket types produce four containers.

```
1 [x*x for x in xs if x > 0]                # list
2 {x*x for x in xs}                         # set, deduplicates
3 {k: v for k, v in pairs}                  # dict
4 (x*x for x in xs)                         # generator, lazy, builds no intermediate list
```

Listing 9: The four forms

The generator form is appropriate when the result is consumed once, for example inside `sum(...)` or `max(...)`, since it avoids materializing a list. Nesting beyond two levels is generally less readable than an explicit loop.

Sorting

- `sorted(xs)` returns a new list. `xs.sort()` sorts in place and returns `None`. The assignment `xs = xs.sort()` binds `None` and fails silently.
- Python's sort is **stable**. Elements comparing equal retain their original relative order. Stability licenses sorting in multiple passes: sorting by the secondary field first and by the primary field second preserves the secondary ordering within each group.
- Sorting a sequence of lists or tuples compares elementwise from the left. The result depends on field order, so an explicit **key** should be supplied whenever the intent is nonobvious.
- Sorting costs $O(n \log n)$, which is negligible at the input sizes these problems use.

The collections module

Four types cover nearly every use. Three are specializations of the dictionary that eliminate the same recurring boilerplate: the check for whether a key already exists before writing to it. The fourth replaces the list when insertion or removal happens at the front.

Counter: frequency. A `Counter` is a dictionary from element to count. It is constructed in one pass over any iterable and answers the question “how many times does each value occur.” Frequency is the most common intermediate object in string and array problems, and constructing one by hand requires an existence check on every element.

```
1 from collections import Counter
2
3 c = Counter("mississippi") # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
4 c['s']                    # 4
5 c['z']                    # 0, a missing key reads as zero and does NOT raise
6
7 c.most_common(2)          # [('i', 4), ('s', 4)], top 2 by count
8 c.most_common()           # every pair, ordered by count descending
9 c.total()                 # sum of all counts
```

Listing 10: Counter

A missing key evaluates to 0 rather than raising `KeyError`, which is what permits `c[x] += 1` to be written unconditionally. The read does not insert the key, so the counter does not grow through inspection.

`most_common(k)` selects the top k with a heap in $O(n \log k)$. Calling `most_common()` with no argument sorts the whole counter at $O(n \log n)$, so the argument should be supplied whenever k is known.

Counters support multiset arithmetic, which expresses several problems directly.

```
1 a, b = Counter("aabbcc"), Counter("abbbd")
2
3 a + b                    # counts added
4 a - b                    # counts subtracted, results <= 0 are dropped
5 a & b                    # intersection: elementwise minimum
6 a | b                    # union: elementwise maximum
```

Listing 11: Counter arithmetic

Two canonical Counter identities

Anagram test. Two strings are anagrams if and only if their counters are equal, so `Counter(s) == Counter(t)` settles the question in $O(n)$. Sorting both strings answers the same question in $O(n \log n)$.

Sliding-window matching. To locate every permutation of a pattern p inside a text t , maintain a counter over a window of length $|p|$ and compare it against `Counter(p)` after each shift. Incrementing the entering character and decrementing the leaving one updates the window in $O(1)$.

defaultdict: grouping. A `defaultdict` is constructed with a zero-argument factory. Accessing a missing key calls that factory, inserts the result, and returns it. The effect is to remove the initialization branch from every accumulation loop.

```
1 from collections import defaultdict
2
3 groups = defaultdict(list)           # missing key -> []
4 for name, dept in employees:
5     groups[dept].append(name)        # no existence check required
6
7 counts = defaultdict(int)           # missing key -> 0
8 adj     = defaultdict(set)          # missing key -> set(), an
    adjacency list
9 grid    = defaultdict(lambda: defaultdict(int)) # a nested, sparse 2D
    table
```

Listing 12: defaultdict

The factory is any callable taking no arguments. `int()` returns 0, `list()` returns [], and `set()` returns an empty set, which is why the bare type name is passed rather than a call to it. A nested default requires a `lambda`, since `defaultdict(defaultdict(int))` would pass an instance where a factory is expected.

`defaultdict(list)` performs a GROUP BY that omits the aggregation step. Every member of each group survives, which is the property that `ROW_NUMBER() OVER (PARTITION BY ...)` supplies in SQL. Building a graph's adjacency list is the same operation applied to edges.

Reading a defaultdict mutates it

Any access to a missing key inserts it. The expression `if d[k]:` therefore creates k with an empty default and the dictionary grows through inspection alone. This corrupts a subsequent iteration over `d.items()` and inflates `len(d)`.

For a read that must not write, use `k in d` or `d.get(k)`. Neither triggers the factory.

The alternative to a `defaultdict` is `d.setdefault(k, []).append(x)` on an ordinary dictionary. It produces the same result and constructs a fresh empty list on every call, including the calls where the key is already present. The `defaultdict` factory runs only on a miss.

deque: a queue. A `deque` is a double-ended queue backed by a doubly linked list of fixed-size blocks. Appending and popping at either end is $O(1)$. The relevant comparison is against a `list`, which stores its elements contiguously, so removing the front element requires shifting every remaining element left by one position.

Operation	list	deque
Append or pop at the right	$O(1)$	$O(1)$
Append or pop at the left	$O(n)$	$O(1)$
Index at position i	$O(1)$	$O(n)$

The costs are complementary. A **list** is the correct stack, since a stack only touches the right end. A **deque** is the correct queue. Random access into a **deque** is a sign that the wrong container was chosen.

```

1 from collections import deque
2
3 q = deque([1, 2, 3])
4 q.append(4)           # right
5 q.appendleft(0)       # left
6 q.pop()              # from the right
7 q.popleft()          # from the left,  $O(1)$  where  $list.pop(0)$  is  $O(n)$ 
8 q.rotate(1)          # shift every element right by one, cyclically

```

Listing 13: deque

Breadth-first search is the standard application. The frontier is processed in insertion order, which requires removal from the front on every iteration of the main loop.

```

1 q = deque([start])
2 seen = {start}
3 while q:
4     node = q.popleft()      # FIFO, so nodes are visited by distance
5     for nxt in adj[node]:
6         if nxt not in seen:
7             seen.add(nxt)
8             q.append(nxt)

```

Listing 14: BFS frontier

Bounded length. A **deque** constructed with `maxlen=k` discards an element from the opposite end whenever a full deque is appended to. This maintains a rolling window over the last k items with no explicit eviction logic, which is useful for moving averages and for any statistic over a fixed lookback.

```

1 w = deque(maxlen=3)
2 for x in stream:
3     w.append(x)           # the oldest element is evicted automatically
4     if len(w) == 3:
5         print(sum(w) / 3)

```

Listing 15: A rolling window of the last 3 values

namedtuple: a labeled record. A tuple carries no information about what its positions mean, so `r[1]` requires the reader to remember the field order. A **namedtuple** attaches names while remaining a tuple, so it stays immutable, hashable, unpackable, and usable as a dictionary key or a set element.

```

1 from collections import namedtuple
2

```

```

3 Student = namedtuple("Student", ["name", "score"])
4 s = Student("Ada", 92.0)
5
6 s.score          # 92.0, and s[1] still works
7 name, score = s  # unpacking is unchanged
8 min(students, key=lambda s: s.score)    # the key function reads
    clearly

```

Listing 16: namedtuple

The cost over a plain tuple is negligible and the gain is that a **key** function or a sort becomes self-documenting. When mutability is required, `dataclasses.dataclass` provides the equivalent with assignable fields.

Iteration

The following forms eliminate manual index arithmetic, which is the usual source of off-by-one errors.

```

1 for i, x in enumerate(xs):          # index and value together
2 for a, b in zip(xs, ys):            # parallel, halts at the shorter
    sequence
3 for a, b in zip(xs, xs[1:]):        # consecutive pairs
4 for x in reversed(xs):              # reverse traversal without index
    arithmetic
5
6 name, score = record                # tuple unpacking
7 first, *rest = xs                   # starred unpacking

```

Listing 17: Iteration forms

The construction `zip(xs, xs[1:])` yields every adjacent pair and underlies run detection, difference sequences, and monotonicity checks. It is the analogue of **LAG** in SQL.

Problem structure

The four-move skeleton

A typical warm-up problem decomposes into four steps.

1. **Parse** the input into a list of records.
2. **Reduce** the records to a target key: a minimum, a maximum, a count, or the k -th distinct value.
3. **Filter** the records against that key.
4. **Order and format** the output.

Step 2 accounts for most errors. The characteristic failure is fusing steps 2 and 3 into a single pass that mishandles an edge case.

Ranking and ties. The most frequent error in step 2 concerns ties. Whether tied elements share a rank or consume one is the same decision in Python as in SQL.

Tie policy	Python	SQL
Ties consume a rank	<code>sorted(xs)[k-1]</code>	<code>ROW_NUMBER() = k</code>
Ties share a rank	<code>sorted(set(xs))[k-1]</code>	<code>DENSE_RANK() = k</code>

For scores [20, 20, 30], the second lowest is 30. Both elements equal to 20 hold rank 1 and neither occupies rank 2, so `sorted(xs)[1]` evaluates to 20 and is incorrect. The `set` converts `ROW_NUMBER` semantics into `DENSE_RANK` semantics.

Rank then filter. Once the key is identified, it is computed in one pass and the records are filtered against it in a second. The structure mirrors the necessity of wrapping a window function in a CTE, since a window function cannot appear in `WHERE`.

```
1 target = sorted({score for _, score in records})[1]    # rank
2 winners = sorted(name for name, score in records if score == target)
3                                     # filter, then order
```

Listing 18: Rank then filter

Common errors

- `sorted(set(xs))[1]` raises `IndexError` when all values are identical.
- Slicing copies. `xs[a:b]` costs $O(b - a)$, so a slice inside a loop makes the loop quadratic.
- `list.insert(0, x)` and `list.pop(0)` are $O(n)$, since every subsequent element shifts. A `deque` provides $O(1)$ operations at both ends.
- Integer division `//` floors toward negative infinity, so `-7 // 2` evaluates to `-4`. `int(-7 / 2)` truncates toward zero.
- A mutable default argument (`def f(xs=[])`) is created once at definition time and persists across calls. The default should be `None`, with the container constructed inside the function body.
- Strings are immutable. Accumulating with `s += c` in a loop is $O(n^2)$. Collect the pieces in a list and apply `".join(parts)"`.

3.3 Algorithmic Paradigms

Each paradigm is identified by a recognition cue in the problem statement.

- **Hashing.** Cue: find a pair, count occurrences, seen before, contains a duplicate. Space is traded for $O(1)$ lookup. The canonical construction tests whether the complement of the current element already appears in a map of previously visited elements.
- **Two pointers.** Cue: a sorted sequence, or a pairing problem. Pointers converging from both ends solve “two elements summing to t ” in $O(n)$ with constant extra space. Fast and slow pointers detect a cycle in a linked list.
- **Sliding window.** Cue: a contiguous subarray or substring subject to a constraint. The right edge expands to admit elements and the left edge contracts while the constraint is violated. Each element enters and leaves once, giving $O(n)$.
- **Prefix sums.** Cue: repeated range-sum queries over a static array. Cumulative sums are precomputed once, after which any range query is a difference of two entries. This is the array analogue of `SUM() OVER`.
- **Binary search on the answer.** Cue: the smallest x such that a condition is feasible. Requires a monotone predicate, meaning that feasibility at x implies feasibility at every larger x . The search ranges over the answer space rather than the input.

- **Sorting as preprocessing.** Cue: intervals, scheduling, or grouping equal elements. Sorting converts a global problem into a local one in which each element need only be compared against its neighbor.

Complexity anchors

Operation	Cost
<code>x in set, x in dict</code>	$O(1)$ average
<code>x in list</code>	$O(n)$
<code>list.append</code>	$O(1)$ amortized
<code>list.insert(0,x), list.pop(0)</code>	$O(n)$
<code>deque.appendleft, deque.popleft</code>	$O(1)$
<code>sorted(xs)</code>	$O(n \log n)$
<code>xs[a:b]</code>	$O(b - a)$, copies

3.4 Recursion and Memoization

A recursive function is defined by three obligations. Omitting any one of them produces either a wrong answer or a stack overflow.

The recursion contract

1. **Base case.** At least one input for which the answer is returned without a recursive call.
2. **Recursive case.** The answer for input n expressed in terms of the answers for strictly smaller inputs.
3. **Progress.** Every recursive call moves the argument closer to a base case, so the recursion terminates.

Correctness follows by induction. Assuming the recursive call returns the correct answer on the smaller input, verify only that the combination step is correct. Tracing the full call tree by hand is unnecessary and is the main reason recursion feels harder than it is.

The call stack. Each pending call occupies a stack frame, so the space cost of a recursion is $O(d)$ where d is the maximum depth. Python imposes a default recursion limit of 1000 frames and raises `RecursionError` beyond it. A recursion whose depth scales with n is therefore unsafe for large n unless it is rewritten as a loop with an explicit stack.

Overlapping subproblems. Naive recursion recomputes identical subproblems. The Fibonacci recurrence $F(n) = F(n-1) + F(n-2)$ generates a call tree with $O(\varphi^n)$ nodes, since $F(n-2)$ is evaluated once by the $F(n-1)$ branch and again directly. Caching each result by its argument collapses the tree to $O(n)$ distinct evaluations.

```
1 from functools import lru_cache
2
3 @lru_cache(maxsize=None)           # or @cache in Python 3.9+
4 def fib(n):
5     if n < 2:                       # base case
6         return n
7     return fib(n-1) + fib(n-2)     # recursive case
```

Listing 19: Memoization by decorator

Arguments to a memoized function must be hashable, which is the reason memoized state is passed as an integer, a string, or a tuple rather than as a list or a set.

Top-down and bottom-up. Memoized recursion is the top-down form. It computes only the states that are reachable, at the cost of a call stack. Tabulation is the bottom-up form. It fills an array in an order that guarantees every dependency is already resolved, using no stack and no recursion.

```
1 def fib(n):
2     if n < 2:
3         return n
4     prev, curr = 0, 1
5     for _ in range(n - 1):
6         prev, curr = curr, prev + curr    # only two states are ever
                                           live
7     return curr
```

Listing 20: The same recurrence, tabulated

The final line illustrates a general reduction. When a recurrence depends only on the previous k states, the table collapses from $O(n)$ space to $O(k)$.

Dynamic programming

Dynamic programming applies when a problem has **optimal substructure**, meaning the optimal solution is composed of optimal solutions to subproblems, together with **overlapping subproblems**, meaning those subproblems recur. Absent overlap, plain divide and conquer suffices. Absent optimal substructure, a greedy or exhaustive method is required.

Formulating a DP

Five questions, answered in order, before any code is written.

1. **State.** What arguments fully determine a subproblem? This is the signature of the function or the index set of the table.
2. **Recurrence.** How is a state's value expressed in terms of smaller states?
3. **Base cases.** Which states are known outright?
4. **Order.** In what order can states be evaluated so that dependencies are resolved first? Top-down recursion answers this automatically.
5. **Answer.** Which state holds the final result?

The complexity is the number of states multiplied by the cost of evaluating one state.

Canonical state definitions. Most problems reuse a small number of state shapes.

State	Meaning	Typical problem
$dp[i]$	best over the prefix ending at i	maximum subarray, house robber
$dp[i][j]$	best over two prefixes	edit distance, longest common subsequence
$dp[i][w]$	first i items, capacity w	0/1 knapsack
$dp[i][j]$	best over the interval $[i, j]$	matrix chain, palindrome partition

Backtracking

Backtracking is recursion over a search tree with an explicit undo step. It enumerates permutations, subsets, and combinations, and prunes branches that cannot yield a valid solution.

```

1 def solve(path, choices, results):
2     if is_complete(path):
3         results.append(path[:])          # copy, since path is mutated in
            place
4     return
5     for c in choices:
6         if not is_valid(path, c):
7             continue                    # prune
8         path.append(c)                  # choose
9         solve(path, remaining(choices, c), results)
10        path.pop()                      # undo

```

Listing 21: Backtracking skeleton

The `path[:]` copy is mandatory. Appending `path` itself stores a reference to a list that subsequent mutation will alter, so every recorded result ends up identical.

For unconstrained enumeration, the standard library is preferable to a hand-written recursion.

```
1 from itertools import permutations, combinations, product
2
3 permutations(xs, r)      # ordered arrangements of length r
4 combinations(xs, r)     # unordered selections of length r
5 product(xs, repeat=r)   # r-tuples with repetition, the r-fold
                        Cartesian power
```

Listing 22: Enumeration without recursion

These correspond exactly to the counting formulas $P(n, r) = n!/(n - r)!$, $\binom{n}{r}$, and n^r .

3.5 Heaps and Priority Queues

A binary heap maintains the minimum of a collection under insertion and deletion at logarithmic cost. Python's `heapq` operates on an ordinary list interpreted as a heap, so no dedicated type is involved.

Operation	Call	Cost
Convert a list in place	<code>heapify(xs)</code>	$O(n)$
Insert	<code>heappush(h, x)</code>	$O(\log n)$
Remove the minimum	<code>heappop(h)</code>	$O(\log n)$
Inspect the minimum	<code>h[0]</code>	$O(1)$
Push then pop	<code>heappushpop(h, x)</code>	$O(\log n)$, one pass

A heap provides no ordering beyond its root. Iterating over the underlying list does not produce sorted output.

Minimum only. `heapq` implements a min-heap exclusively. A max-heap is obtained by negating the keys on insertion and negating again on extraction. For records, push the tuple `(-priority, item)`, since tuples compare lexicographically and the first field dominates.

Top- k under a stream

To retain the k largest of n elements, maintain a min-heap of size k . Each new element is compared against the root, which is the smallest of the current k . If the element is larger, the root is evicted and the element inserted.

The cost is $O(n \log k)$ and the memory is $O(k)$. Sorting the full input costs $O(n \log n)$ and requires all n elements in memory at once, so the heap is superior when $k \ll n$ and is the only option when the input arrives as a stream of unknown length.

```
1 import heapq
2
3 def top_k(stream, k):
4     h = []                                     # min-heap holding the k best
5     so far
6     for x in stream:
7         if len(h) < k:
8             heapq.heappush(h, x)
9         elif x > h[0]:                         # h[0] is the weakest survivor
10             heapq.heapreplace(h, x)          # pop the root, then push x
```

```
return sorted(h, reverse=True)
```

Listing 23: The k largest elements

When the whole input already fits in memory, the library provides this directly as `heapq.nlargest(k, xs)` and `heapq.nsmallest(k, xs)`, both of which accept a `key` argument.

Two heaps for a running median. Maintain a max-heap over the lower half of the values and a min-heap over the upper half, rebalancing after each insertion so their sizes differ by at most one. The median is then read from the root of the larger heap, or is the mean of the two roots when the sizes are equal. Insertion costs $O(\log n)$ and the median query is $O(1)$.

TODO: Add graph traversal (BFS, DFS, and the `deque`-based queue) alongside the heap entry, since Dijkstra composes the two. Add a string-processing section covering `split`, `join`, and sliding windows over characters.

Style Reference Page

Definition 3.1 (Expectation). For a discrete random variable X with support $\mathcal{X}$, $\mathbb{E}[X] = \sum_{x \in \mathcal{X}} x \mathbb{P}(X = x)$, provided the sum converges absolutely.

Linearity of expectation

For any random variables $X_1, \dots, X_n$ and scalars a_i ,

$$\mathbb{E} \left[\sum_{i=1}^n a_i X_i \right] = \sum_{i=1}^n a_i \mathbb{E}[X_i].$$

This holds with no independence assumption. It is the single highest-yield identity for interview brainteasers, especially when combined with indicator variables $\mathbf{1}\{\cdot\}$.

Theorem 3.2 (Variance of a sum). If $X_1, \dots, X_n$ are pairwise uncorrelated, then $\text{Var}(\sum_i X_i) = \sum_i \text{Var}(X_i)$.

Proof. Expand $\text{Var}(\sum_i X_i) = \sum_i \text{Var}(X_i) + \sum_{i \neq j} \text{Cov}(X_i, X_j)$. Pairwise uncorrelatedness kills every cross term, leaving the claim. $\square$

Example 3.3. Draw a uniformly random permutation of $\{1, \dots, n\}$. Let X count fixed points. Write $X = \sum_i \mathbf{1}\{\pi(i) = i\}$. Each indicator has mean $1/n$, so by linearity $\mathbb{E}[X] = 1$, independent of n .

TODO: Add the variance derivation and the derangement limit $\mathbb{P}(X = 0) \rightarrow e^{-1}$.

Remark. Keep a running list of distributions with their mean, variance, and MGF. Interviewers probe recall speed on Bernoulli, Binomial, Geometric, Poisson, Exponential, and Normal.